Dio Errors, Status Codes and Client-Server Communication

1. Dio Error Types (DioExceptionType)

When a request fails, Dio throws a **DioException**. Its **type** field tells you why it failed.

Type	Meaning	What to do
connectionTimeout	Could not connect to the server in time.	Show "check your connection", allow retry.
sendTimeout	Sending the request (e.g. uploading data) took too long.	Retry, or check upload size.
receiveTimeout	Server did not finish sending the response in time.	Retry, or increase the timeout.
badCertificate	The server's HTTPS certificate is not trusted.	Do not ignore it. Report as a security error.
badResponse	Server replied, but with an error status code (4xx / 5xx).	Read e.response?.statusCode and handle it (see part 2).
cancel	The request was cancelled with a CancelToken.	Usually ignore (user left the screen).
connectionError	No connection at all (no internet, wrong host, DNS failure).	Show "no internet connection".
unknown	Any other error (e.g. parsing problem).	Log e.error and show a generic message.

Example of handling:

```
try {
  final res = await dio.get('/users');
  return res.data;
} on DioException catch (e) {
  switch (e.type) {
    case DioExceptionType.connectionTimeout:
    case DioExceptionType.sendTimeout:
    case DioExceptionType.receiveTimeout:
      throw 'Request timed out, try again';
    case DioExceptionType.connectionError:
      throw 'No internet connection';
    case DioExceptionType.badResponse:
      final code = e.response?.statusCode;
      if (code == 401) throw 'Please log in again';
      if (code == 404) throw 'Not found';
      if (code != null && code >= 500) throw 'Server error, try later';
      throw 'Request failed ($code)';
    case DioExceptionType.cancel:
      throw 'Request cancelled';
    default:
      throw 'Something went wrong';
  }
}
```

Note: unlike fetch() in JavaScript, Dio throws on 4xx and 5xx by default (controlled by the validateStatus option).

2. HTTP Status Codes

1xx = info, 2xx = success, 3xx = redirect, 4xx = client mistake, 5xx = server problem.

Code	Name	Meaning / frontend action
200	OK	Request worked, response has data.
201	Created	New resource created (usually after POST).
204	No Content	Worked, but no body (common after DELETE).
301 / 302	Moved / Found	Resource is at another URL (redirect).
304	Not Modified	Use the cached copy.
400	Bad Request	Invalid data sent. Show validation errors.
401	Unauthorized	Not logged in, or token missing / expired. Refresh token or go to login.
403	Forbidden	Logged in, but not allowed. Show "no permission".
404	Not Found	Resource or route does not exist.
405	Method Not Allowed	Wrong method (e.g. POST on a GET-only route).
409	Conflict	Clashes with existing data (e.g. email already used).
422	Unprocessable Entity	Validation failed (form errors).
429	Too Many Requests	Rate limited. Wait and retry.
500	Internal Server Error	Bug on the server. Show a generic error.
502	Bad Gateway	A server in the middle got a bad reply from another server.
503	Service Unavailable	Server down or overloaded. Retry later.
504	Gateway Timeout	A server in the middle waited too long for another server.

401 vs 403: 401 means "who are you?" (bad or missing token). 403 means "I know who you are, but you are not allowed".

3. How Client and Server Communicate

Request / Response

The client (app or browser) sends a **request**, the server sends back a **response**. Every exchange follows this pattern.

Request contains	Response contains
Method: GET, POST, PUT, PATCH, DELETE	Status code: 200, 404, 500 ...
URL: https://api.example.com/users/42	Headers: Content-Type, Cache-Control ...
Headers: Authorization, Content-Type ...	Body: the data (usually JSON)
Body: data sent (for POST / PUT / PATCH)	

Steps of one request

- **DNS:** the domain name (api.example.com) is turned into an IP address.
- **TCP connection:** client and server open a connection.
- **TLS (HTTPS):** the connection is encrypted and the server's certificate is checked.
- **HTTP request** is sent, the server processes it (may read the database).
- **HTTP response** comes back with a status code and body.

Stateless communication

HTTP is stateless: the server does not remember earlier requests. In a REST API every request must carry everything needed to understand it (who you are, what you want). This lets any server handle any request, which makes scaling easy.

Authentication with JWT

- Client sends username and password to the auth server (POST /login).
- Server checks them in the database and creates a JWT: header.payload.signature.
- The payload (user id, role, expiry) is only encoded, so anyone can read it. Never put secrets in it.
- The signature is made with the server's private key, so nobody else can create a valid token.
- Client sends the token with every request: `Authorization: Bearer <token>`.
- Any server with the public key can verify the token without a database or session lookup.
- Access tokens are short-lived; a refresh token is used to get a new one.

Session vs JWT

Session (stateful)	JWT (stateless)
Server stores a session record after login.	Server stores nothing after login.
Cookie carries only an ID (a pointer).	Token carries the user data itself, signed.
Every request needs a lookup in the session store.	Every request is verified with the key only.
Easy to log out / revoke.	Hard to revoke before it expires.

Server to server

Servers talk to each other the same way: one server acts as the client and sends an HTTP request to another (for example an order service calling a payment service). They pass the JWT or a service token along so the other server can verify who is calling. If the second server fails or is slow, the first one may return 502 or 504 to the app.